

PROJET FIL ROUGE · CDA 2026

Amoura

Application web de restaurant
Réservation & fidélité

JALON 2 – FÉVRIER 2026

Méthodologie & Organisation du Projet
Conception UI/UX

Amoura

Formation CDA (Concepteur Développeur d'Applications)
Dépôt Git : [github.com/\[votre-username\]/amoura](https://github.com/[votre-username]/amoura)

Table des matières

1. Contexte et rappel du projet	3
2. Méthodologie de gestion de projet	3
2.1 Méthode retenue – Scrum adapté solo	3
2.2 Planning macro – Diagramme de Gantt	4
2.3 Outils de suivi et de collaboration	4
3. Gestion du code source – Git	5
3.1 Stratégie de branches (GitFlow adapté)	5
3.2 Conventions de commits	5
4. Pipeline CI/CD – Stratégie DevOps	6
4.1 Intégration Continue (CI)	6
4.2 Livraison Continue (CD)	6
5. Conception UI/UX	7
5.1 Sitemap et arborescence	7
5.2 Charte graphique	7
5.3 Maquettes desktop & mobile	8
5.4 Considérations UX	9

1. Contexte et rappel du projet

Ce document constitue le livrable du **Jalon 2** du projet fil rouge CDA. Il couvre deux volets distincts tels que définis dans le cahier des charges technique :

- ◆ La **méthodologie et l'organisation** du projet (gestion de projet, planning, Git, CI/CD)
- ◆ La **conception UI/UX** (sitemap, charte graphique, maquettes desktop et mobile)

Le projet **Amoura** consiste à développer une application web de gestion de réservations et de programme de fidélité pour un restaurant méditerranéen fictif, dans le cadre de la formation CDA. La stack technique retenue est : **Symfony (API REST) + React + MariaDB + Docker**, avec authentification JWT.

2. Méthodologie de gestion de projet

2.1 Méthode retenue – Scrum adapté solo

La méthode **Scrum adaptée** a été retenue pour piloter ce projet individuel. Les raisons principales :

- ◆ Les **6 jalons mensuels** correspondent naturellement à des sprints d'une durée fixe
- ◆ Scrum favorise les **livraisons itératives** et l'ajustement continu du backlog
- ◆ Adapté au contexte solo : les cérémonies (daily, rétrospective) sont remplacées par un **journal d'avancement hebdomadaire**

Rôle Scrum	Occupant	Responsabilités
Product Owner	Étudiant (moi)	Définir et prioriser le backlog, valider les US
Scrum Master	Étudiant (moi)	Lever les obstacles, veiller au respect du process
Developer	Étudiant (moi)	Réaliser les tâches du sprint, livrer les incrément
Stakeholder	Formateur	Valider les jalons, donner des retours

2.2 Planning macro – Diagramme de Gantt

Le tableau ci-dessous synthétise la répartition des jalons sur les 6 mois du projet. Le jalon en cours est mis en évidence.

Jalon	Période	Livrables principaux	Statut
J1 – Janvier	Jan 2026	Cahier des charges fonctionnel (CDCF)	✓ Livré

J2 – Février	Fév 2026	Méthodologie + Maquettes UI/UX	→ En cours
J3 – Mars	Mar 2026	Modélisation BDD (MCD / MLD / MPD)	À venir
J4 – Avril	Avr 2026	Conception UML + Architecture multi-couches	À venir
J5 – Mai	Mai 2026	Dev bêta + Sécurité + Tests automatisés	À venir
J6 – Juin	Juin 2026	Déploiement + Mise en production + Rapport final	À venir

Note : un diagramme de Gantt détaillé (tâches hebdomadaires) peut être produit via un outil dédié (TeamGantt, Excel) et fourni en annexe à la demande du formateur.

2.3 Outils de suivi et de collaboration

Les outils suivants sont utilisés pour l'organisation quotidienne du projet :

- ◆ **GitHub Projects (Kanban)** : tableau À faire / En cours / Terminé / Bloqué
- ◆ **GitHub Issues** : chaque fonctionnalité ou user story est tracée comme une issue avec labels (feature, bug, doc, security)
- ◆ **README.md** du dépôt : documentation d'installation, lancement Docker, comptes de test
- ◆ **Journal d'avancement** : note hebdomadaire dans le wiki du repo (réalisé / bloqué / plan)

La priorisation du backlog suit la méthode **MoSCoW** : Must Have (fonctionnalités core – réservation, fidélité), Should Have (statistiques admin), Could Have (prototype cliquable Figma), Won't Have (paiement en ligne).

3. Gestion du code source – Git

3.1 Stratégie de branches (GitFlow adapté)

Le dépôt Git suit une **stratégie GitFlow simplifiée** adaptée au développement solo :

Branche	Rôle	Politique
main	Versions stables (livrables jalons)	Protégée – merge uniquement depuis develop via PR
develop	Intégration continue des features	Merge après validation des tests CI
feature/*	Nouvelles fonctionnalités	Créée depuis develop, fusionnée après revue
fix/*	Correctifs de bugs	Créée depuis develop ou main selon criticité
release/*	Préparation releases	Créée depuis develop pour finalisation de jalon

Le dépôt est hébergé sur **GitHub** (privé, accès formateur accordé). La branche main est protégée : aucun push direct n'est autorisé, les merge requests vers main nécessitent que la CI soit au vert.

3.2 Conventions de commits

Les messages de commit suivent la convention **Conventional Commits** pour faciliter la génération automatique du changelog et la lisibilité de l'historique :

Préfixe	Signification	Exemple
feat:	Nouvelle fonctionnalité	feat(reservation): add availability check endpoint
fix:	Correction de bug	fix(auth): prevent JWT refresh loop on 401
docs:	Documentation	docs: update README with Docker run instructions
test:	Ajout/modification de tests	test(loyalty): add unit tests for points calculator
chore:	Maintenance, dépendances	chore: upgrade Symfony 6.4 → 7.0
refactor:	Refactoring sans effet	refactor(booking): extract availability service
ci:	Configuration CI/CD	ci: add PHPUnit step to GitHub Actions workflow

4. Pipeline CI/CD – Stratégie DevOps

4.1 Intégration Continue (CI)

La CI est configurée via **GitHub Actions**. Un fichier `.github/workflows/ci.yml` déclenche automatiquement les vérifications suivantes à chaque push sur develop et à chaque Pull Request :

Étape CI	Outil	Déclencheur	Description
Lint PHP	PHP CS Fixer	Push develop / PR	Vérification du style de code PSR-12
Lint JS/TS	ESLint	Push develop / PR	Contrôle du code React (règles Airbnb)
Tests unitaires	PHPUnit	Push develop / PR	Couverture des services métier (cible ≥ 80%)
Tests API	PHPUnit Functional	Push develop / PR	Tests des endpoints Symfony (statuts HTTP, JSON)
Build Docker	Docker Buildx	Push develop	Construction et validation de l'image PHP/Nginx
Analyse statique	PHPStan (niveau 5)	Push develop	Détection des erreurs de typage sans exécution

Le badge CI (build: passing / failing) sera visible dans le README du dépôt dès la mise en place effective prévue courant mars (Jalon 3).

4.2 Livraison Continue (CD)

Le volet CD sera implémenté progressivement :

- ◆ **Jalon 3-4 (mars-avril)** : Publication automatique de l'image Docker sur **Docker Hub** à chaque merge sur develop
- ◆ **Jalon 5 (mai)** : Déploiement automatique sur un environnement de préproduction (PaaS, ex. Railway ou Render – gratuit)
- ◆ **Jalon 6 (juin)** : Pipeline complète : build → test → push image → deploy prod

Même si le déploiement sur serveur réel n'est pas encore effectif à ce stade, **la stratégie CI/CD est planifiée et documentée**. L'infrastructure cible est containerisée (docker-compose) avec variables d'environnement gérées via les secrets GitHub Actions.

5. Conception UI/UX

5.1 Sitemap et arborescence

L'application est structurée autour de **8 pages principales** accessibles selon le rôle de l'utilisateur :

Page	URL	Accès	Description
Accueil	/	Public	Présentation du restaurant, hero, CTA réservation
Menu / Carte	/menu	Public	Carte du restaurant par catégorie
Notre histoire	/about	Public	Histoire, valeurs, équipe
Contact	/contact	Public	Coordonnées, formulaire, carte Google Maps
Réservation	/reservation	Connecté	Calendrier, créneaux, confirmation
Connexion / Inscription	/login	Public	Authentification JWT, Google OAuth
Profil & Fidélité	/profil	Connecté	Points, réservations, badges
Administration	/admin/*	Admin	Dashboard, gestion réservations, clients, fidélité

5.2 Charte graphique

L'identité visuelle d'Amoura est construite autour d'un positionnement **raffiné et méditerranéen**. Le contraste noir/crème évoque l'élégance d'un établissement gastronomique, tandis que l'or apporte la chaleur et le caractère premium.

Nom	Hex	Usage
Crème	#faf8f5	Fond principal des pages, textes sur fonds sombres
Noir	#0a0a0a	Fonds sombres, textes principaux, boutons CTA
Or	#c9a96e	Accents, logo, prix, éléments premium, fidélité
Warm Dark	#1a1410	Hero, sections sombres chaudes, sidebar admin
Gris clair	#e8e5e0	Séparateurs, fonds alternatifs, bordures

Typographie

Cormorant Garamond (serif, Google Fonts) — Titres, logo, noms de plats, prix, citations. Graisses utilisées : Light (300), Regular (400), Regular Italic. Ce choix évoque la tradition gastronomique française.

Montserrat (sans-serif, Google Fonts) — Navigation, labels, corps de texte, formulaires. Graisses utilisées : Light (300), Regular (400), Medium (500). Apporte lisibilité et modernité en complément du serif.

5.3 Maquettes desktop & mobile

Les maquettes ont été réalisées en **HTML/CSS interactif** (livrable maquettes_amoura.html) et présentent les **8 pages** de l'application en format desktop (1280px) et mobile (375px). Le fichier permet de naviguer entre les pages et de basculer entre les deux formats.

Page maquetée	Format	Éléments clés illustrés
Accueil	Desktop + Mobile	Hero plein écran, CTA réservation, section chef, footer
Menu	Desktop + Mobile	Grille plats, filtres par catégorie, prix, allergènes
Notre histoire	Desktop + Mobile	Split image/texte, valeurs, équipe
Contact	Desktop + Mobile	Informations + formulaire + placeholder Google Maps
Réservation	Desktop + Mobile	Calendrier pas-à-pas, créneaux, récapitulatif, fidélité
Connexion	Desktop + Mobile	Split visual/form, onglets connexion/inscription, OAuth
Profil & Fidélité	Desktop + Mobile	Barre de points, badges, historique réservations
Administration	Desktop + Mobile	Sidebar, KPIs, tableau réservations, chart fréquentation

5.4 Considérations UX

Les choix UX suivants ont guidé la conception de l'interface :

- ◆ **Approche Mobile First** : les maquettes mobiles ont été conçues en priorité, avec une navigation par barre de bas d'écran (bottom nav) et des gestes tactiles natifs
- ◆ **Parcours réservation en 3 étapes** : date → créneau → personnes, avec récapitulatif persistant pour éviter les erreurs de saisie
- ◆ **Feedback immédiat** : états actif/sélectionné/survol sur tous les éléments interactifs (calendrier, créneaux, boutons)
- ◆ **Accessibilité** : contraste WCAG AA respecté sur tous les textes principaux (ratio ≥ 4.5:1), navigation au clavier envisagée pour le développement
- ◆ **Intégration du programme de fidélité** dans le tunnel de réservation : rappel des points gagnables pour inciter à la connexion
- ◆ **Google OAuth** : login simplifié pour réduire la friction à l'inscription

- ◆ **Cohérence des composants** : navbar, footer, boutons CTA, formulaires — styles réutilisables définis dans la charte CSS

Ce document sera mis à jour au fil du projet pour refléter les évolutions de conception. Les maquettes interactives (maquettes_amoura.html) constituent le livrable visuel principal de ce Jalon 2.